CommonJS Modules

10 min read Updated 24 Jul 2026 Topic 11 of 70

Why it matters

Every non-trivial Node program is split across many files, and CommonJS (CJS) is the mechanism that has glued those files together since Node's first release in 2009. When you type `require('express')` or `module.exports = router`, you are using CommonJS. Even in 2025, with ES Modules fully supported, the overwhelming majority of packages on npm still ship CommonJS, and the default for a `.js` file with no `"type"` declaration is still CommonJS. If you cannot explain what `require`, `module.exports`, and `exports` actually do, you will hit bugs that feel like magic: a value that is mysteriously `undefined`, a module that runs twice, or a circular dependency that returns half an object.

CommonJS is also a favourite interview topic precisely because it looks trivial but hides real machinery: a wrapper function, a resolution algorithm, and a cache. Understanding it separates people who copy-paste `require` lines from people who can debug a production module-loading problem at 2 a.m.

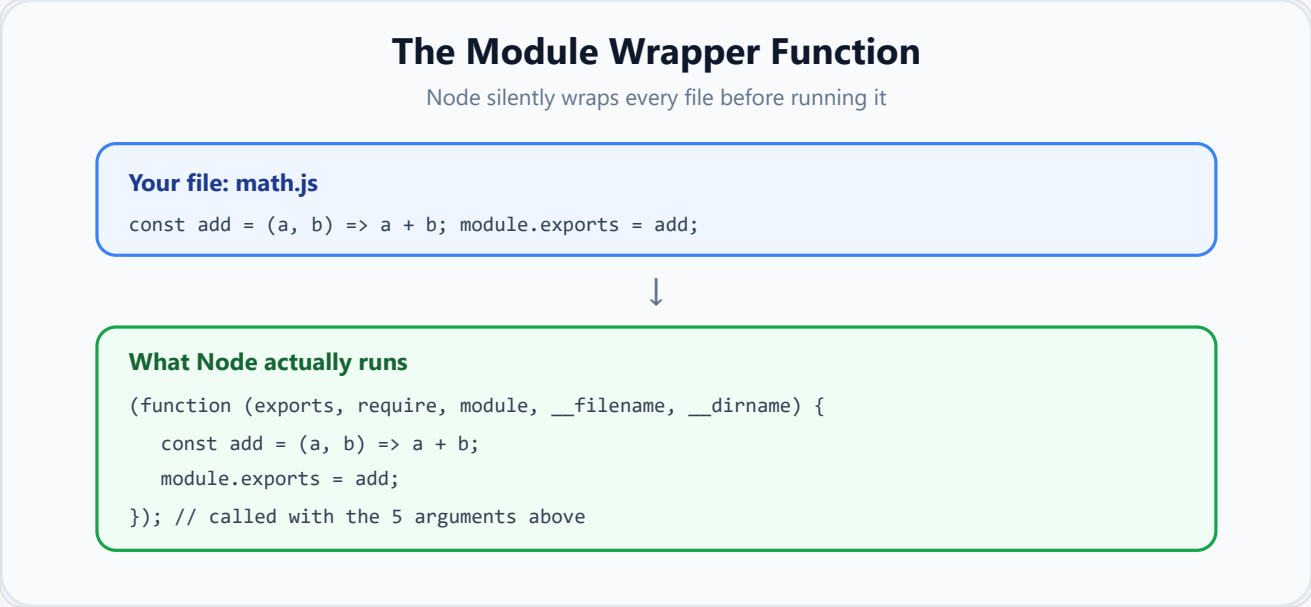
The concept, simply

A CommonJS module is just a file. Each file gets its own private scope — variables you declare at the top level of a file are *not* global, they belong to that file. To share something with other files, you attach it to a special object called `module.exports`. To pull in what another file shared, you call `require('./that-file')`, which returns whatever that file assigned to its `module.exports`.

Two directions, two keywords:

- **Import** — `const x = require('./file')` reads that published value synchronously.

The reason each file has a private scope is not a language feature of JavaScript — it is a trick Node plays. Before running your file, Node wraps its source in a function. That single fact explains almost everything surprising about CommonJS.



Node wraps every module's source in a function before executing it, which is why top-level variables stay private and why exports, require, module, __filename and __dirname are magically available.

How it works

The module wrapper. Node never runs your file's source directly. It concatenates it into a function that looks exactly like this:

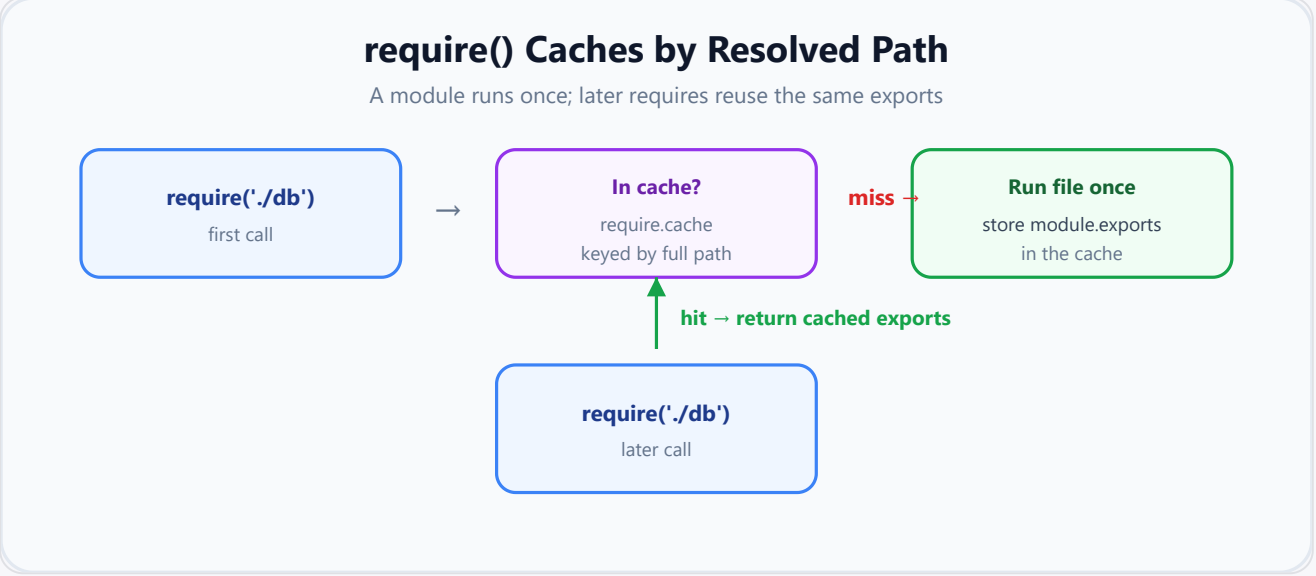
JAVASCRIPT Copy

```
(function (exports, require, module, __filename, __dirname) {
  // ... your file's code goes here ...
});
```

Node then calls that function with the five arguments. This is why `require`, `module`, `exports`, `__filename`, and `__dirname` appear "out of nowhere" in every file — they are ordinary function parameters, not globals. It is also why a `const` declared at the top of a file does not leak: it is a local variable inside that wrapper function. You can even

module.exports vs exports. When the wrapper runs, Node sets up `module.exports = {}` and then does `exports = module.exports`. So `exports` starts as a reference to the same empty object. Mutating that object works for both: `exports.add = ...` and `module.exports.add = ...` touch the same object. But *reassigning* `exports = someNewThing` only rebinds the local parameter — it no longer points at `module.exports`, so nothing is exported. The value that is actually returned to the caller is always `module.exports`. Rule of thumb: to export one thing (a class, a function), write `module.exports = X`; to export several named things, attach them with `exports.name = ...` or `module.exports.name = ...`.

The resolution + cache flow. When you call `require(spec)`, Node runs the module resolution algorithm to turn `spec` into a single absolute file path (covered in depth in the module-resolution topic). It then checks `require.cache`, an object keyed by that resolved absolute path. If an entry exists, Node returns its `.exports` immediately — the file is **not** re-run. If there is no entry, Node creates a new `Module` object, stores it in the cache *before* executing (this is what lets circular dependencies terminate), wraps and runs the file, and finally returns `module.exports`.



The first require of a path runs the file and stores its exports; every later require of the same resolved path returns the cached object without re-running the module — modules are effectively singletons.

Caching means modules are singletons. Because the cache is keyed by resolved path, requiring the same module ten times from ten files gives every file the exact same

need a fresh evaluation you must delete the cache entry (`delete require.cache[require.resolve('./x')]`) — a hack used in test tooling and hot-reloaders, almost never in application code.

__dirname and __filename. `__filename` is the absolute path of the current file; `__dirname` is the directory containing it. Both are computed by Node per file and injected via the wrapper. They are essential for building paths that work regardless of the process's current working directory — always prefer `path.join(__dirname, 'data.json')` over a bare relative string, because `fs` resolves bare relative paths against `process.cwd()`, not against the file.

require is synchronous. `require` reads and executes the target file on the spot, blocking until it returns. That is acceptable because it happens at startup, but it is also why `require` cannot load ES Modules (which may need asynchronous, top-level `await`) in older Node versions — bridging the two worlds needs the async `import()` instead.

Code examples

1. Exporting several named values.

JAVASCRIPT Copy

```
// math.js
function add(a, b) { return a + b; }
function sub(a, b) { return a - b; }

exports.add = add;          // same as module.exports.add
module.exports.sub = sub;   // both attach to the one exports object

// app.js
const math = require('./math');
console.log(math.add(2, 3)); // 5
const { sub } = require('./math'); // destructure named exports
console.log(sub(9, 4));      // 5
```

2. Exporting a single thing (the classic gotcha).

```
// logger.js  - WRONG
exports = function log(msg) { console.log(msg); };
// reassigns the local 'exports' param; module.exports is still {}

// logger.js  - RIGHT
module.exports = function log(msg) { console.log(msg); };

// app.js
const log = require('./logger');
log('hello'); // works only with the RIGHT version
```

3. The cache makes modules singletons.

JAVASCRIPT Copy

```
// counter.js
let count = 0;
module.exports = {
  inc() { return ++count; },
  get() { return count; },
};

// a.js
const c = require('./counter');
c.inc(); c.inc();

// b.js
const c = require('./counter');
console.log(c.get()); // 2 — same instance as a.js, count is shared
```

4. Reading a file next to the module, safely.

JAVASCRIPT Copy

```
const fs = require('node:fs');
const path = require('node:path');

// Robust: resolves against THIS file, not the process cwd.
const configPath = path.join(__dirname, 'config.json');
```

Common mistakes & gotchas

- **Reassigning `exports`.** `exports = fn` silently exports nothing. Only `module.exports = fn` works for single-value exports.
- **Expecting a fresh module each require.** The cache returns the same object; top-level side effects run once. Do not put per-request state at module scope.
- **Assuming top-level `const` is global.** It is scoped to the file's wrapper — that is by design, and it is why one file cannot see another file's locals.
- **Bare relative paths in `fs`.** `fs.readFileSync('config.json')` resolves against `process.cwd()`, so it breaks when the app is started from another directory. Use `__dirname`.
- **Circular dependencies returning a partial object.** If A requires B while B is still requiring A, B receives A's *incomplete* `module.exports` (whatever was assigned so far). Move the `require` below the code that finishes populating exports, or restructure.
- **Mutating another module's exports object.** Because it is a shared singleton, changing a property leaks to every other requirer. Treat imported objects as read-only.
- **Requiring inside a hot loop.** After the first call it is a cache lookup, but it still costs a resolution hash and clutters code — require at the top of the file.

Best practices

- Put all `require` calls at the top of the file so dependencies are obvious and load order is predictable.
- Prefer `module.exports = { a, b, c }` (one object literal) over many `exports.x =` lines — it reads as a clear public API and avoids the `exports`-reassignment trap.
- Use the `node:` prefix for builtins (`require('node:fs')`) so it is unambiguous a core module is meant and never shadowed by a package named `fs`.

- Keep top-level module code cheap and side-effect-light; heavy work (opening connections) should be a function you call, so the cost is explicit.
- Always build filesystem paths with `path.join(__dirname, ...)`.
- Avoid `delete require.cache[...]` in application code — it is a testing/tooling hack that leads to memory leaks and duplicate singletons.
- Break circular dependencies by extracting the shared piece into a third module both can require.

⭐ Interview lens

Q: What is the module wrapper function and why does it exist? Node wraps each file's source in `(function (exports, require, module, __filename, __dirname) { ... })` and calls it. It gives every file a private scope (top-level vars are locals, not globals) and injects the five module-scoped identifiers as parameters. This is why `require` and `__dirname` "just exist" without being imported.

Q: Difference between `module.exports` and `exports`? They start pointing at the same object (`exports = module.exports = {}`). Mutating properties on either works. But only `module.exports` is returned to the caller, so reassigning `exports` to a new value breaks the export while reassigning `module.exports` works. Trap: interviewers show `exports = function(){}` and ask what the caller receives — answer: an empty object `{}`.

Q: Is `require` synchronous or asynchronous, and where is the result stored? Synchronous — it reads and executes the file immediately, blocking. Results are cached in `require.cache`, keyed by the module's resolved absolute path, so the file runs at most once per process.

module in the cache *before* executing it. If A→B→A, the second require of A returns A's partially-built exports (whatever existed at that moment), not a crash and not the finished object. This can yield `undefined` for members assigned later. Fix by reordering requires or extracting shared code.

Q: How do you make a module NOT be a singleton? You can't cleanly — the cache guarantees a singleton per resolved path. Options: export a factory function callers invoke to get fresh instances, or (hack) delete the cache entry. The idiomatic answer is "export a factory."

Q: Why is `path.join(__dirname, 'x.json')` safer than `'./x.json'` for `fs`? `fs` resolves relative paths against `process.cwd()` (where `node` was launched), not the file's location. `__dirname` is always the file's own directory, so the path is correct no matter where the process started.

Cheat-card

- **Wrapper:** `(exports, require, module, __filename, __dirname) ⇒ {}` — private scope + injected identifiers.
- **Export one:** `module.exports = X`. **Export many:** `exports.a = ...`.
- **Never** `exports = X` — rebinds the param, exports nothing.
- **require** is synchronous; result cached in `require.cache` by resolved absolute path.
- **Modules are singletons** — top-level code runs once per process.
- **Circular dep** → partial exports, not a crash (cached before execution).
- **__dirname** = file's dir, **__filename** = file's path; use `path.join(__dirname, ...)`.
- Prefix builtins: `require('node:fs')`.

QUICK CHECK — no marks, just for you.

1. How do `exports`, `__filename`, and `__dirname` "appear out of nowhere" in a CommonJS file?

- ◯ They are true JavaScript globals
- ◯ They are imported automatically from `node:core`
- ◯ Node wraps each file's source in a function and passes them as parameters
- ◯ The V8 engine injects them into the global object

Check answer

2. What does the CommonJS require cache guarantee about a module?

- ◯ It is re-executed on every `require` call
- ◯ A fresh copy is returned to each caller
- ◯ It is never cached unless you opt in
- ◯ It is effectively a singleton — the file runs once and every requirer gets the same exports object

Check answer

3. Why does `exports = function log(msg){...}` fail to export the function while `module.exports = ...` works?

- ◯ Reassigning `exports` only rebinds the local parameter; only `module.exports` is returned to the caller
- ◯ `exports` can only hold objects, never functions
- ◯ `exports` is read-only and throws on assignment

Check answer

Previous
Blocking the Event Loop

Next
ES Modules

LEARN MORE

ARTICLES & DOCS

- Modules: CommonJS modules
  nodejs.org
- JavaScript modules (import / export)
  developer.mozilla.org

VIDEOS



Node JS Tutorial for Beginners #6 - Modules and require()
Net Ninja



How to Import and Export Modules in Node JS (CommonJS)
Nikita Dev

Hand-picked references for this lesson.